برنامه‌نویسی با کمک هوش مصنوعی: از ایده تا پیاده‌سازی

هوش مصنوعی شیوه برنامه‌نویسی را تغییر داده است. امروزه می‌توانیم از ابزارهای هوشمند برای طراحی راه‌حل، تولید کد، توضیح برنامه‌ها، پیدا کردن خطاها و حتی ساخت بخش‌های مختلف یک نرم‌افزار استفاده کنیم.

اما استفاده مؤثر از هوش مصنوعی به معنای سپردن تمام کارها به آن نیست؛ بلکه یعنی بتوانیم مسئله را درست تعریف کنیم، اطلاعات لازم را در اختیارش قرار دهیم و نتیجه کار را بررسی کنیم.

در این مبحث یاد می‌گیریم چگونه با کمک هوش مصنوعی از یک ایده به برنامه‌ای قابل‌اجرا برسیم.

۱. روش‌های استفاده از هوش مصنوعی در برنامه‌نویسی

هوش مصنوعی را می‌توان در مراحل مختلف توسعه نرم‌افزار به کار گرفت:

- درک مسئله: بررسی ایده، شناسایی نیازها و پیشنهاد راه‌حل.
- طراحی برنامه: تعیین اجزای برنامه و ارتباط میان آن‌ها.
- تولید کد: نوشتن توابع، کلاس‌ها و بخش‌های مختلف برنامه.
- یادگیری: توضیح مفاهیم، بررسی مثال‌ها و درک کدهای ناآشنا.
- اشکال‌زدایی: تحلیل خطاها و پیشنهاد راه‌حل.
- آزمایش و بهبود: پیشنهاد تست، بررسی حالت‌های مختلف و افزایش کیفیت کد.
- مستندسازی: تهیه توضیحات، راهنمای استفاده و مستندات برنامه.

لازم نیست همیشه از هوش مصنوعی بخواهیم یک برنامه کامل بنویسد. گاهی درخواست توضیح یک خطا یا نوشتن یک تابع کوچک، نتیجه دقیق‌تر و مفیدتری دارد.

۲. تفاوت چت‌بات، دستیار کدنویسی و ابزارهای عامل‌محور

ابزارهای هوش مصنوعی در برنامه‌نویسی یکسان نیستند. شناخت تفاوت آن‌ها به ما کمک می‌کند برای هر کار، ابزار مناسبی انتخاب کنیم.

چت‌بات‌های هوش مصنوعی

چت‌بات‌ها ابزارهایی گفت‌وگومحور هستند که می‌توانیم مسئله را برایشان توضیح دهیم، درباره راه‌حل‌ها سؤال کنیم، کد بگیریم یا کد موجود را برای بررسی به آن‌ها بدهیم.

مناسب برای: یادگیری پایتون، بررسی ایده‌ها، تولید نمونه کد، توضیح خطاها و طراحی اولیه برنامه.

دستیارهای کدنویسی

این ابزارها برای کمک به نوشتن کد طراحی شده‌اند و معمولاً در ویرایشگرهای برنامه‌نویسی یا محیط توسعه ادغام می‌شوند. آن‌ها می‌توانند کد پیشنهاد دهند، تکمیل کنند یا بخش‌هایی از کد موجود را اصلاح کنند.

مناسب برای: نوشتن و تکمیل کد هنگام برنامه‌نویسی، دریافت پیشنهاد در متن کد و اصلاح سریع بخش‌های کوچک.

ابزارهای توسعه عامل‌محور (Agentic Coding)

این ابزارها می‌توانند با دریافت یک هدف، چند مرحله از کار را برنامه‌ریزی و اجرا کنند. بسته به امکانات و مجوزهایشان، ممکن است فایل‌ها را بخوانند و تغییر دهند، چندین فایل را ویرایش کنند، فرمان اجرا کنند و نتیجه تست‌ها را بررسی کنند.

مناسب برای: انجام وظایف چندمرحله‌ای، تغییر ساختار پروژه، پیاده‌سازی قابلیت‌های نسبتاً بزرگ و اجرای فرایندهای تکراری توسعه.

تفاوت اصلی این است که چت‌بات معمولاً در قالب گفت‌وگو راهنمایی می‌کند، دستیار کدنویسی در جریان نوشتن کد کمک می‌کند و ابزار عامل‌محور می‌تواند بخشی از فرایند توسعه را نیز اجرا کند.

البته مرز میان این دسته‌ها همیشه مشخص نیست و بعضی ابزارها قابلیت‌های هر سه گروه را ترکیب می‌کنند.

چگونه ابزار مناسب را انتخاب کنیم؟

کار نوع	مناسب انتخاب
پایتون مفهوم یک یادگیری	بات‌چت
کد قطعه یک توضیح	کدنویسی دستیار یا بات‌چت
ویرایشگر در تابع یک تکمیل	کدنویسی دستیار
کوچک برنامه یک اولیه نمونه ساخت	کدنویسی دستیار یا بات‌چت
پروژه یک در فایل چند تغییر	محورعامل ابزار یا پیشرفته دستیار
هاتست بررسی و توسعه مرحله چند اجرای	محورعامل ابزار

برای پروژه‌های ساده آموزشی، یک چت‌بات معمولاً کافی است. هرچه پروژه بزرگ‌تر و چندمرحله‌ای‌تر شود، ابزارهایی که به فایل‌ها و محیط توسعه دسترسی دارند، می‌توانند مفیدتر باشند.

ابزار پیچیده‌تر لزوماً انتخاب بهتری نیست. ابزار باید با اندازه پروژه، مهارت برنامه‌نویس و میزان کنترلی که بر فرایند لازم است، تناسب داشته باشد.

۳. تبدیل ایده به دستورالعمل قابل‌اجرا

یکی از مهم‌ترین مهارت‌ها در کار با هوش مصنوعی، تبدیل یک خواسته کلی به دستورالعملی روشن و قابل‌اجراست.

فرض کنید می‌خواهیم برنامه‌ای برای مدیریت هزینه‌های روزانه بسازیم.

درخواست مبهم:

> یک برنامه مدیریت هزینه با پایتون بنویس.

این درخواست جزئیات کافی ندارد. مشخص نیست اطلاعات چگونه وارد شوند، کجا ذخیره شوند یا برنامه چه امکاناتی داشته باشد.

درخواست بهتر:

> با پایتون برنامه‌ای ساده برای ثبت هزینه‌های روزانه بنویس. کاربر بتواند عنوان هزینه و مبلغ آن را وارد کند، هزینه‌ها در یک فایل متنی ذخیره شوند و در پایان مجموع هزینه‌ها نمایش داده شود. فعلاً از کتابخانه‌های استاندارد پایتون استفاده کن و برنامه را در قالب یک فایل قابل‌اجرا بنویس. کد را ساده نگه دار و بخش‌های اصلی آن را توضیح بده.

در درخواست دوم، هدف، امکانات اصلی، محدودیت‌ها و شکل خروجی مشخص‌تر هستند.

برای تبدیل ایده به دستورالعمل، به این پرسش‌ها پاسخ دهید:

1. هدف چیست؟ برنامه قرار است چه مسئله‌ای را حل کند؟
2. کاربر چه کاری انجام می‌دهد؟ چه اطلاعاتی وارد می‌کند؟
3. خروجی چیست؟ برنامه باید چه نتیجه‌ای تولید کند؟
4. محدودیت‌ها چیست؟ از چه زبان، کتابخانه یا محیطی باید استفاده شود؟
5. معیار موفقیت چیست؟ از کجا می‌فهمیم برنامه درست کار می‌کند؟

هرچه پاسخ این پرسش‌ها روشن‌تر باشد، احتمال دریافت نتیجه مناسب بیشتر می‌شود.

۴. اصول پرامپت‌نویسی برای تولید کد

به دستور یا پیامی که برای هدایت هوش مصنوعی ارسال می‌کنیم، پرامپت (Prompt) می‌گوییم.

پرامپت خوب لزوماً طولانی نیست؛ مهم این است که اطلاعات لازم را به‌شکل روشن منتقل کند.

هنگام درخواست تولید کد، بهتر است موارد زیر را مشخص کنیم:

- نقش یا نوع کمک موردنیاز: مثلاً می‌خواهیم کد تولید شود یا کد موجود بررسی شود.
- هدف: چه مسئله‌ای باید حل شود؟

- ورودی و خروجی: برنامه چه داده‌هایی می‌گیرد و چه نتیجه‌ای می‌دهد؟
- محدودیت‌ها: از چه زبان، کتابخانه‌ها یا روش‌هایی استفاده شود؟
- سطح پیچیدگی: کد آموزشی و ساده باشد یا برای یک پروژه بزرگ طراحی شود؟
- معیارهای بررسی: چه شرایطی باید رعایت شوند و چه حالت‌هایی باید آزمایش شوند؟

برای مثال، به‌جای درخواست «کدی برای محاسبه میانگین بنویس»، می‌توانیم بنویسیم:

> تابعی در پایتون به نام `calculate_average` بنویس که یک لیست از اعداد را دریافت کند و میانگین آن‌ها را برگرداند. اگر لیست خالی بود، رفتار تابع را به‌صورت روشن مشخص کن. از کتابخانه خارجی استفاده نکن و دو مثال برای آزمایش تابع ارائه بده.

در این درخواست، نام تابع، نوع ورودی، خروجی مورد انتظار، یک حالت خاص و محدودیت استفاده از کتابخانه‌ها مشخص شده‌اند.

نکته مهم: در برنامه‌نویسی، حالت‌های مرزی را فراموش نکنید؛ مانند ورودی خالی، مقدار منفی، داده نامعتبر یا فایلی که وجود ندارد. این حالت‌ها از منابع رایج خطا هستند.

۵. ارائه زمینه کافی به هوش مصنوعی (Context Engineering)

هوش مصنوعی فقط بر اساس درخواست آخر ما پاسخ نمی‌دهد؛ اطلاعاتی که درباره پروژه، کدهای موجود، محدودیت‌ها و تصمیم‌های قبلی در اختیارش قرار می‌دهیم نیز بر نتیجه اثر می‌گذارند.

به انتخاب و سازمان‌دهی این اطلاعات برای هدایت بهتر هوش مصنوعی، مهندسی زمینه (Context Engineering) می‌گوییم.

فرض کنید از هوش مصنوعی می‌خواهیم قابلیت جدیدی به یک برنامه اضافه کند. اگر فقط بگوییم «قابلیت ثبت کاربر را اضافه کن»، ممکن است کدی تولید کند که با ساختار فعلی برنامه سازگار نباشد.

برای ارائه زمینه مناسب، می‌توانیم اطلاعات زیر را در اختیارش بگذاریم:

- هدف کلی پروژه و کاربران آن.
- زبان برنامه‌نویسی و کتابخانه‌های مورد استفاده.
- ساختار فایل‌ها و مسئولیت هر فایل.
- کدهای مرتبط با قابلیت موردنظر.
- رفتار فعلی برنامه و مشکلی که باید حل شود.
- محدودیت‌ها و تصمیم‌هایی که قبلاً گرفته‌ایم.
- نمونه ورودی، خروجی یا خطای مشاهده‌شده.

برای مثال:

> این پروژه با پایتون نوشته شده و اطلاعات هزینه‌ها را در یک فایل JSON ذخیره می‌کند. تابع `load_expenses()` مسئول خواندن اطلاعات است. می‌خواهم امکان حذف یک هزینه را اضافه کنم. ساختار فعلی داده‌ها را حفظ کن، از کتابخانه خارجی استفاده نکن و فقط فایل‌ها و توابع مرتبط را تغییر بده. اگر برای تصمیم‌گیری اطلاعات کافی نداری، ابتدا سؤال بپرس.

لازم نیست در هر درخواست تمام پروژه را دوباره توضیح دهیم. باید اطلاعات مرتبط با همان وظیفه را انتخاب کنیم.

زمینه بیشتر همیشه به معنای نتیجه بهتر نیست. اطلاعات نامرتبط، قدیمی یا متناقض ممکن است باعث سردرگمی شوند. هدف، ارائه اطلاعات کافی و دقیق است.

همچنین اگر ابزار به فایل‌های پروژه دسترسی دارد، باید بررسی کنیم که واقعاً فایل‌ها و بخش‌های مرتبط را خوانده است؛ نباید صرفاً فرض کنیم همه اطلاعات را در اختیار دارد.

۶. تقسیم پروژه به وظایف کوچک و قابل‌مدیریت

درخواست ساخت یک نرم‌افزار کامل در یک مرحله معمولاً انتخاب مناسبی نیست؛ زیرا ممکن است جزئیات مهم نادیده گرفته شوند یا بخش‌های مختلف با یکدیگر سازگار نباشند.

بهتر است پروژه را به وظایف کوچک‌تر تقسیم کنیم.

برای مثال، ساخت برنامه مدیریت هزینه را می‌توان به این مراحل تقسیم کرد:

1. مشخص‌کردن اطلاعاتی که هر هزینه باید داشته باشد.
2. نوشتن تابع ثبت هزینه.
3. ذخیره هزینه‌ها در فایل.
4. خواندن هزینه‌های ذخیره‌شده.
5. محاسبه مجموع هزینه‌ها.
6. اضافه‌کردن امکان نمایش یا حذف هزینه‌ها.
7. آزمایش برنامه و اصلاح خطاها.

هر وظیفه باید نتیجه مشخصی داشته باشد و تا حد امکان بتوان آن را جداگانه بررسی کرد.

برای هر مرحله می‌توانیم از هوش مصنوعی بخواهیم: - راه‌حل را پیشنهاد دهد. - کد همان بخش را تولید کند. - روش اجرای آن را توضیح دهد. - چند آزمایش مناسب پیشنهاد کند.

پس از بررسی نتیجه، به مرحله بعد می‌رویم.

این روش کمک می‌کند خطاها زودتر شناسایی شوند و اگر مشکلی ایجاد شد، پیدا کردن علت آن آسان‌تر باشد.

۷. تولید کد به‌صورت مرحله‌ای

پس از تقسیم پروژه، بهتر است کد را نیز مرحله‌به‌مرحله تولید کنیم.

یک فرایند ساده می‌تواند چنین باشد:

مرحله اول: طراحی

از هوش مصنوعی بخواهید راه‌حل پیشنهادی و اجزای اصلی برنامه را توضیح دهد. اگر مسئله مبهم است، پیش از تولید کد درباره جزئیات تصمیم بگیرید.

مرحله دوم: پیاده‌سازی

از آن بخواهید فقط بخش مشخصی از برنامه را بنویسد؛ مثلاً تابع ثبت هزینه.

مرحله سوم: اجرا

کد را در محیط خود اجرا کنید و خروجی را بررسی کنید.

مرحله چهارم: آزمایش

ورودی‌های معمول و حالت‌های مرزی را امتحان کنید. بررسی کنید که آیا نتیجه با انتظار شما مطابقت دارد.

مرحله پنجم: اصلاح

اگر مشکلی وجود داشت، پیام خطا، کد مرتبط و نتیجه مورد انتظار را در اختیار هوش مصنوعی قرار دهید.

مرحله ششم: ادامه توسعه

پس از اطمینان از عملکرد بخش فعلی، به سراغ بخش بعدی بروید.

در هر مرحله باید بدانیم چه چیزی تغییر کرده و چرا. این روش از تولید حجم زیادی کد که هنوز درستی آن بررسی نشده است، جلوگیری می‌کند.

۸. دریافت توضیح درباره کدهای تولیدشده

یکی از بهترین کاربردهای هوش مصنوعی، کمک به فهم کد است.

اگر کدی دریافت کردیم که کاملاً آن را درک نمی‌کنیم، بهتر است پیش از استفاده از آن، درباره بخش‌های مهمش سؤال کنیم.

برای مثال:

> این تابع پایتون را توضیح بده. ابتدا هدف کلی آن را بیان کن، سپس ورودی‌ها، خروجی و مراحل اصلی اجرا را توضیح بده. اگر بخشی از کد به مفهوم خاصی در پایتون وابسته است، آن مفهوم را ساده توضیح بده.

برای بررسی دقیق‌تر می‌توانیم بپرسیم:

- این متغیر چه نقشی دارد؟

- چرا از این تابع یا کتابخانه استفاده شده است؟
- اگر ورودی خالی باشد چه اتفاقی می‌افتد؟
- این خطا چگونه ایجاد می‌شود؟
- آیا می‌توان همین کار را ساده‌تر انجام داد؟
- چگونه می‌توان این تابع را آزمایش کرد؟

بهتر است توضیح را به بخش‌های کوچک تقسیم کنیم و اگر نکته‌ای نامفهوم بود، درباره همان قسمت سؤال بپرسیم.

معیار مهم این است که بتوانیم منطق اصلی کد را خودمان توضیح دهیم. اگر فقط کد را کپی کنیم، ممکن است در مواجهه با خطا یا نیاز به تغییر، نتوانیم مسئله را حل کنیم.

۹. اصلاح کد با کمک هوش مصنوعی

وقتی برنامه خطا دارد یا نتیجه درستی تولید نمی‌کند، هوش مصنوعی می‌تواند در پیدا کردن علت و پیشنهاد اصلاح کمک کند.

اما درخواست‌هایی مانند «کدم کار نمی‌کند، درستش کن» معمولاً اطلاعات کافی ندارند.

برای دریافت راهنمایی بهتر، این اطلاعات را ارائه دهید:

1. کد مرتبط با مشکل.
2. پیام خطا، در صورت وجود.
3. ورودی‌ای که مشکل را ایجاد می‌کند.
4. خروجی مورد انتظار.
5. خروجی واقعی یا رفتار نادرست برنامه.

برای مثال:

> این تابع باید میانگین اعداد یک لیست را محاسبه کند، اما هنگام دریافت لیست خالی خطا می‌دهد. کد و پیام خطا را بررسی کن. ابتدا علت مشکل را توضیح بده، سپس کمترین تغییر لازم را پیشنهاد کن و دو آزمایش برای بررسی اصلاح انجام‌شده بنویس.

این درخواست کمک می‌کند به‌جای بازنویسی کامل برنامه، روی مشکل مشخص تمرکز کنیم.

پس از دریافت اصلاح پیشنهادی:

- کد را بخوانید و تغییرات را بررسی کنید.
- آن را در محیط خود اجرا کنید.
- حالت ایجادکننده خطا را دوباره آزمایش کنید.
- مطمئن شوید اصلاح جدید، رفتارهای قبلی برنامه را خراب نکرده است.

اگر مشکل حل نشد، نتیجه اجرای جدید را در اختیار هوش مصنوعی قرار دهید و فرایند را ادامه دهید.

هدف، فقط حذف پیام خطا نیست؛ باید علت مشکل را بفهمیم و مطمئن شویم اصلاح انجام‌شده درست است.

۱۰. شناخت محدودیت‌ها و خطاهای هوش مصنوعی

هوش مصنوعی ابزار مفیدی است، اما ممکن است کدی تولید کند که ظاهراً درست به نظر برسد و در عمل خطا داشته باشد.

برخی از محدودیت‌های رایج آن عبارت‌اند از:

- کد نادرست: کد ممکن است خطای نحوی یا منطقی داشته باشد.
- استفاده از امکانات ناموجود: گاهی تابع یا قابلیتی پیشنهاد می‌شود که در کتابخانه موردنظر وجود ندارد یا نحوه استفاده از آن اشتباه است.
- نادیده گرفتن حالت‌های خاص: ممکن است کد فقط برای ورودی‌های معمول درست کار کند.
- ناسازگاری با پروژه: کد تولیدشده ممکن است با ساختار یا نسخه کتابخانه‌های پروژه هماهنگ نباشد.
- پیچیدگی غیرضروری: گاهی راه‌حل پیشنهادی بیش از اندازه پیچیده است.
- مشکلات امنیتی: کد ممکن است اطلاعات حساس را افشا کند، ورودی‌ها را به‌درستی بررسی نکند یا عملیات خطرناکی انجام دهد.

بنابراین، کد تولیدشده توسط هوش مصنوعی باید مانند کد نوشته‌شده توسط هر برنامه‌نویس دیگری بررسی و آزمایش شود.

برای ارزیابی کد، به این موارد توجه کنید:

- آیا کد اجرا می‌شود؟
- آیا خروجی آن برای ورودی‌های مختلف درست است؟
- آیا خطاهای احتمالی مدیریت شده‌اند؟
- آیا اطلاعات حساس یا فایل‌های مهم در معرض خطر قرار می‌گیرند؟
- آیا کد از کتابخانه‌ها و روش‌های مناسب استفاده می‌کند؟
- آیا خودتان می‌توانید منطق اصلی آن را توضیح دهید؟

در پروژه‌های واقعی، به‌ویژه پروژه‌هایی که با اطلاعات کاربران، حساب‌ها یا داده‌های حساس سروکار دارند، بررسی امنیت و کیفیت کد اهمیت بیشتری پیدا می‌کند.

همچنین هرگز صرفاً به دلیل پیشنهاد هوش مصنوعی، فرمان ناشناخته‌ای را اجرا نکنید یا اجازه دسترسی نامحدود به فایل‌ها و اطلاعات حساس ندهید.

۱۱. یک الگوی ساده برای کار مؤثر با هوش مصنوعی

برای بیشتر تمرین‌ها و پروژه‌های آموزشی، می‌توانیم از الگوی زیر استفاده کنیم:

1. تعریف مسئله: دقیقاً مشخص کنید چه چیزی می‌خواهید بسازید.
2. ارائه زمینه: اطلاعات مرتبط، کدهای موجود و محدودیت‌ها را توضیح دهید.
3. تقسیم کار: وظیفه را به مراحل کوچک تقسیم کنید.
4. درخواست راه‌حل: برای هر مرحله، خروجی مشخص بخواهید.
5. درک کد: بخش‌های مهم و تصمیم‌های اصلی را بررسی کنید.
6. اجرا و آزمایش: کد را در محیط خود اجرا کنید.
7. اصلاح: خطاها را با ارائه اطلاعات دقیق بررسی کنید.
8. بازبینی نهایی: عملکرد، خوانایی و ایمنی برنامه را ارزیابی کنید.

این الگو یک فرایند خشک و اجباری نیست. برای کارهای کوچک می‌توان مراحل را ترکیب کرد و برای پروژه‌های بزرگ، هر مرحله را با جزئیات بیشتری انجام داد.

نکته اصلی این است که هوش مصنوعی باید بخشی از فرایند حل مسئله باشد، نه جایگزین فهم مسئله و بررسی نتیجه.

جمع‌بندی

برای استفاده مؤثر از هوش مصنوعی در برنامه‌نویسی، باید بتوانیم مسئله را روشن تعریف کنیم، پرامپت مناسبی بنویسیم، زمینه کافی ارائه دهیم و پروژه را به وظایف کوچک تقسیم کنیم.

چت‌بات‌ها، دستیارهای کدنویسی و ابزارهای عامل‌محور قابلیت‌های متفاوتی دارند و انتخاب میان آن‌ها باید بر اساس نوع کار و میزان پیچیدگی پروژه انجام شود.

در نهایت، تولید کد تنها بخشی از کار است. فهم کد، اجرای آن، آزمایش حالت‌های مختلف، اصلاح خطاها و بررسی امنیت نیز اهمیت دارند.

هدف این دوره آن است که بتوانیم با کمک هوش مصنوعی سریع‌تر و مؤثرتر برنامه بسازیم، بدون آنکه کنترل و درک خود را از فرایند توسعه از دست بدهیم.

تمرین

می‌خواهیم با پایتون برنامه‌ای ساده برای مدیریت هزینه‌های روزانه بسازیم. برنامه باید بتواند هزینه جدید ثبت کند، اطلاعات را در فایل ذخیره کند و مجموع هزینه‌ها را نمایش دهد.

این تمرین را به مراحل زیر تقسیم کنید:

1. از هوش مصنوعی بخواهید نیازهای برنامه را به چند وظیفه کوچک تقسیم کند.
2. برای مرحله اول، پرامپتی بنویسید که تولید یک تابع برای ثبت هزینه را درخواست کند.
3. کد پیشنهادی را اجرا کنید و با چند ورودی مختلف آزمایش کنید.

4. از هوش مصنوعی بخواهید کد را توضیح دهد و حالت‌های مرزی احتمالی را مشخص کند.
5. اگر خطایی پیدا کردید، کد، پیام خطا و خروجی مورد انتظار را ارائه دهید و اصلاح بخواهید.
6. پس از اصلاح، کد را دوباره اجرا کنید و نتیجه را بررسی کنید.

تمرین تکمیلی: از دو ابزار متفاوت برای حل یک وظیفه یکسان استفاده کنید. خروجی‌ها را از نظر درستی، سادگی، خوانایی و سهولت آزمایش مقایسه کنید. سپس توضیح دهید کدام راه‌حل را انتخاب می‌کنید و چرا.